

Actividades Práctico-Experimentales

Nro. 001

1. Datos Generales

Asignatura	Estructura de Datos
Ciclo	3ro A / B
Unidad	1: Estructuras de datos estáticas y dinámicas
Resultado de aprendizaje de la unidad	Modela el uso de memoria y diseña Tipos de Datos Abstractos (TDA) para comprender las diferencias de complejidad espacial, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	01
Título de la Práctica	Micro-TDAs, Abstracción de Datos y Análisis de Memoria en Motores V8
Nombre del Docente	Cristian Ramiro Narváez Guillén
Nombre de los integrantes	Isaac Vire Danny Beltran Axel Jimenez Lucas Orellana Noelia Bustan
Fecha	Jueves 01 y Viernes 02 de octubre 2026
Horario	Jueves 01 de octubre 10:30 – 11:30 Viernes 02 de octubre 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

- Modelar Tipos de Datos Abstractos (TDA) simples utilizando clases y constructores en JavaScript moderno.

- Medir empíricamente el impacto en la memoria Heap (complejidad espacial $S(n)$) al procesar estructuras de datos basadas en tipos primitivos frente a arreglos de objetos masivos.
- Evaluar mediante el análisis del recolector de basura (Garbage Collector) las decisiones de diseño físico y su impacto en la sostenibilidad computacional (Green Computing).

3. Materiales y Reactivos

- Software Node.js (versión LTS 20.x o superior) instalado en el sistema.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación técnica oficial (ABI).
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Procedimiento / Metodología

Enfoque PID-2025:

- ABP (Aprendizaje Basado en Problemas): El equipo es contratado por un proyecto de geolocalización satelital sostenible. Deben cargar $5N = 10^6$ coordenadas en memoria. El problema radica en que instanciar 10^6 objetos colapsa los límites del entorno. Se debe diseñar y medir la alternativa más eficiente.
- ABI (Aprendizaje Basado en Investigación): Los estudiantes deberán investigar cómo el motor de JavaScript (V8) reserva memoria contigua vs. memoria dinámica.
- Modalidad: Individual y discusión grupal posterior.

Tarea 1: Preparación del Entorno de Medición (Profiler)

Para medir la huella de memoria en JavaScript, utilizaremos el módulo integrado process.

1. Cree un nuevo directorio para el proyecto y abra su editor de texto.
2. Cree un archivo denominado profiler.js.
3. Implemente la función de medición de memoria RAM (Heap):

```
/**
```

```
 * Tarea 1: Utilidad para medir la memoria Heap en Megabytes (MB).
```

```
 * El Heap es el espacio de memoria dinámico donde V8 almacena objetos y variables.
```

```
 */
```

```
function getMemoryUsage() {
```

```
    const memoryData = process.memoryUsage();
```

```
    // Convertimos de bytes a Megabytes (MB) usando notación matemática estándar
```

```
    const heapUsedMB = Math.round(memoryData.heapUsed / 1024 / 1024 * 100)
```

```

/ 100;
    return heapUsedMB;
}

module.exports = { getMemoryUsage };

```

Tarea 2: Enfoque 1 - Modelado basado en Arreglo de Objetos (TDA)

En esta tarea, crearemos N objetos. Esta es la representación de más alto nivel de abstracción.

1. Cree un archivo llamado lab01_objetos.js.
2. Incluya el siguiente código, el cual creará 1,000,000 de instancias de coordenadas.

```

const { getMemoryUsage } = require('./profiler');

const N = 1000000; // 1 millón de registros
const memoriaInicial = getMemoryUsage();

// TDA: Coordenada basada en Objetos
class CoordenadaObj {
    constructor(lat, lng) {
        this.lat = lat; // Número de punto flotante de 64 bits
        this.lng = lng;
    }
}

// Almacenamos en memoria
let coordenadas = [];
for (let i = 0; i < N; i++) {
    coordenadas.push(new CoordenadaObj(i * 0.1, i * -0.1));
}

const memoriaFinal = getMemoryUsage();
console.log(`[Enfoque Objetos] Memoria inicial: ${memoriaInicial} MB`);
console.log(`[Enfoque Objetos] Memoria final: ${memoriaFinal} MB`);
console.log(`[Enfoque Objetos] Consumo Neto: ${memoriaFinal - memoriaInicial} MB`);

3. Ejecute en su terminal: node lab01_objetos.js y anote el resultado neto.

```

Tarea 3: Enfoque 2 - Modelado basado en Arreglos de Primitivos paralelos (Data-Oriented Design)

1. Cree un archivo llamado lab01_primitivos.js.
2. Implemente la estructura de datos desacoplando la abstracción del objeto a arreglos paralelos unidimensionales fuertemente tipados (Float64Array).

```
const { getMemoryUsage } = require('./profiler'); const N =

1000000;

// TDA Desacoplado: Uso de Typed Arrays para obligar a V8 a usar memoria contigua
// La capacidad total requerida es N elementos de 8 bytes (64 bits) cada uno let lat = new
Float64Array(N);
let lng = new Float64Array(N);

    for (let i = 0; i < N; i++) {
        lat[i] = i * 0.1;
        lng[i] = i * -0.1;
    }
const memoriaFinal = getMemoryUsage();
console.log(`[Enfoque Primitivos] Memoria inicial: ${memoriaInicial} MB`);
console.log(`[Enfoque Primitivos] Memoria final: ${memoriaFinal} MB`); console.log(`[Enfoque
Primitivos] Consumo Neto: ${memoriaFinal - memoriaInicial} MB`);
3. Ejecute en su terminal: node lab01_primitivos.js y anote el resultado.
```

Tarea 4: Investigación y Cálculo de Complejidad

Basado en los resultados, el estudiante debe aplicar ABI para investigar el porqué de la diferencia matemática.

1. Busque en la documentación oficial de V8 cómo se empaquetan las propiedades en los *Hdi den Clas ses* de JavaScript.
2. Calcule teóricamente en LaTeX el espacio asintótico. Sabiendo que un Float64 pesa 8 bytes, demuestre por qué.

5. Resultados obtenidos del grupo:

Resultados de los códigos de las tareas:

Tarea 1

```
JS profiler.js > ...
1  /**
2   * El Heap es el espacio de memoria dinámico donde V8 almacena objetos y
3   variables.
4   */
5
6   function getMemoryUsage() {
7       const memoryData = process.memoryUsage();
8       const heapUsedMB = Math.round(memoryData.heapUsed / 1024 / 1024 * 100) / 100;
9       return heapUsedMB;
10  }
11
12  module.exports = { getMemoryUsage };
```

Tarea 2

```

JS lab01_objetos.js > ...
1  const { getMemoryUsage } = require('./profiler');
2  const N = 1000000;
3  const memoriaInicial = getMemoryUsage();
4
5  class CoordenadaObj {
6    constructor(lat, lng) {
7      this.lat = lat;
8      this.lng = lng;
9    }
10 }
11 let coordenadas = [];
12 for (let i = 0; i < N; i++) {
13   coordenadas.push(new CoordenadaObj(i * 0.1, i * -0.1));
14 }
15 const memoriaFinal = getMemoryUsage();
16 console.log(`[Enfoque Objetos] Memoria inicial: ${memoriaInicial} MB`);
17 console.log(`[Enfoque Objetos] Memoria final: ${memoriaFinal} MB`);
18 console.log(`[Enfoque Objetos] Consumo Neto: ${memoriaFinal - memoriaInicial} MB`);
19

```

```

[Enfoque Objetos] Memoria inicial: 4.09 MB
[Enfoque Objetos] Memoria final: 95.08 MB
[Enfoque Objetos] Consumo Neto: 90.99 MB

```

Tarea 3

```

const { getMemoryUsage } = require('./profiler');
const N = 1000000;
const memoriaInicial = getMemoryUsage();

let lat = new Float64Array(N);
let lng = new Float64Array(N);
for (let i = 0; i < N; i++) {
  lat[i] = i * 0.1;
  lng[i] = i * -0.1;
}
const memoriaFinal = getMemoryUsage();
console.log(`[Enfoque Primitivos] Memoria inicial: ${memoriaInicial} MB`);
console.log(`[Enfoque Primitivos] Memoria final: ${memoriaFinal} MB`);
console.log(`[Enfoque Primitivos] Consumo Neto: ${memoriaFinal - memoriaInicial} MB`);

```

```

[Enfoque Primitivos] Memoria inicial: 4.09 MB
[Enfoque Primitivos] Memoria final: 4.45 MB
[Enfoque Primitivos] Consumo Neto: 0.36000000000000003 MB

```

Tabla de comparación entre la tarea 2 y 3:

Enfoque	Heap Inicial	Heap Final	Consumo Neto	Número de ejecuciones
Primitivos	4.23 MB	4.53 MB	0.30 MB	5

Objetos	4.23 MB	94.43 MB	90.19 MB	5
---------	---------	----------	----------	---

La razón del cambio de consumo de memoria Heap es por el hecho de que esta está enfocada en guardar información de los objetos, sus clases ocultas y su espacio en memoria y almacenamiento. Por otro lado, los primitivos no son objetos, por lo que su almacenamiento en bits recae en un ArrayBuffer.

Resultado de la tarea 4:

- **¿Cómo se empaquetan las propiedades en los Hidden Classes de JavaScript?**

Las propiedades en las Hidden Classes se gestionan por medio de un DescriptorArray, el cual está referenciado en el Map, una clase oculta que se almacena en el primer puntero del objeto en memoria y guarda la forma de este. En este DescriptorArray se guarda el nombre de cada propiedad con nombre, sus atributos y su posición. La forma en que se empaquetan los valores depende del tipo de propiedad:

Propiedades nombradas

Pueden empaquetarse directamente dentro del mismo objeto, en un almacén de propiedades externo al que se accede mediante el índice indicado en el DescriptorArray cuando se agota el espacio en el objeto, o mediante un diccionario de propiedades. Este último modo se activa al agregar y eliminar propiedades dinámicamente de forma constante, por lo que permite agregar y borrar propiedades, pero ralentiza el acceso a las mismas.

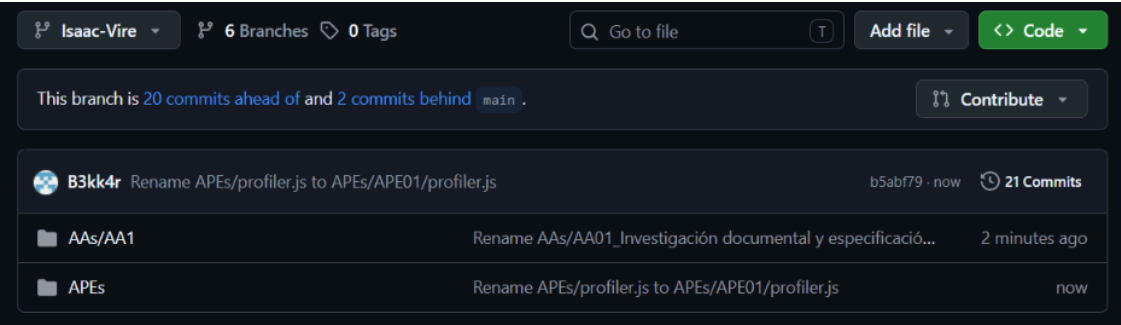
Propiedades indexadas

Utilizadas comúnmente en arrays, no se registran en el DescriptorArray del Map, ya que se almacenan en un almacén de elementos separados. Se organizan según su densidad y uso: compactas o con huecos, según si existen posiciones no definidas, y rápidas o de diccionario, donde en las rápidas el índice de las propiedades coincide con el índice de almacenamiento y las de diccionario donde se emplean en arreglos muy dispersos para ahorrar memoria.

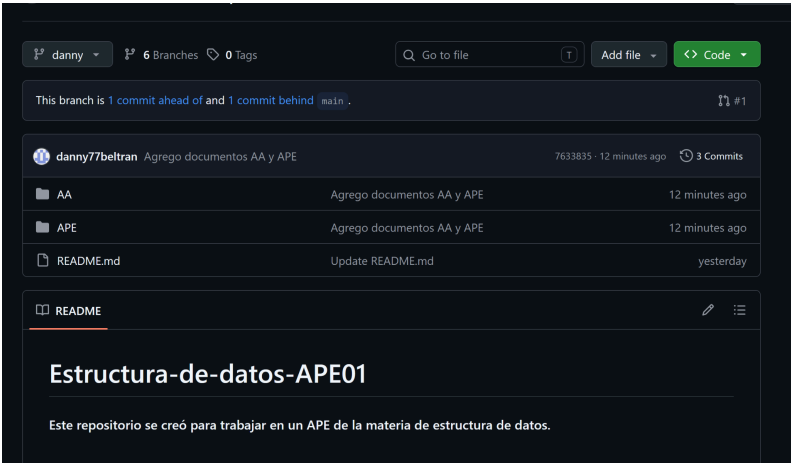
Entonces, cada instancia de CoordenadaObj posee dos propiedades nombradas lat y lng que se asignan siempre en el mismo orden, por lo que el millón de objetos comparte un único Map, por la naturaleza de la estructura TransitionArray de las HiddenClasses, y un único DescriptorArray. Aun así, hay un costo por instancia, y los campos de tipo double se almacenan como HeapNumber, de modo que cada objeto contiene su cabecera (puntero al Map) y referencias a dos HeapNumber donde residen realmente los valores. Pues como un valor de punto flotante no puede almacenarse directamente en un campo etiquetado, se lo guarda en un objeto HeapNumber dentro del heap y el campo del objeto contiene una referencia a él. Por el contrario, un Float64Array almacena cada valor como 8 bytes contiguos, sin cabecera, sin punteros y sin objetos intermedios.

Evidencias del repositorio:

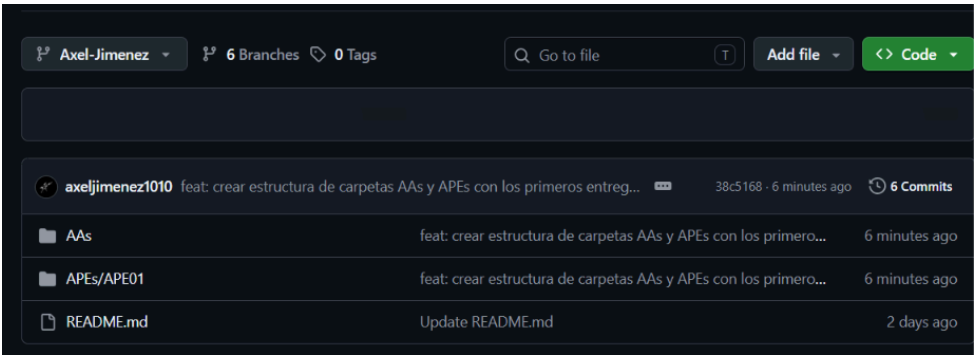
Isaac Vire



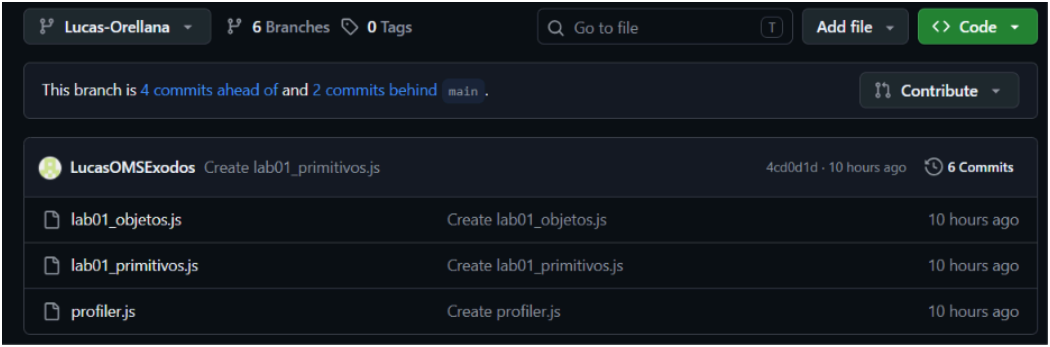
Danny Beltrán



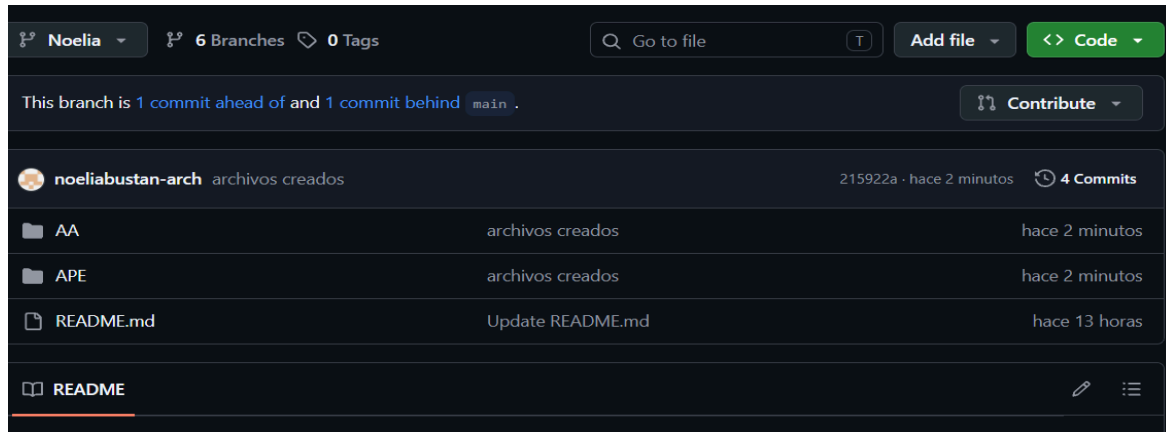
Axel



Lucas



Noelia



6. Preguntas de Control:

Responda las siguientes preguntas reflexionando sobre las causas fundamentales de los fenómenos computacionales observados:

1. Causa Material: ¿De qué está constituido fundamentalmente un objeto en la memoria del entorno de ejecución (V8), y en qué difiere la "materia" de un objeto de la de un valor primitivo continuó en un Float64Array?

Dentro del entorno de ejecución V8, un objeto de JavaScript está constituido principalmente por un encabezado de metadatos y punteros a sus almacenes de datos, mientras que la “materia” de un valor dentro de Float64Array es un dato binario numérico puro que se almacena en memoria contigua.

Un objeto en el heap está compuesto por tres palabras de encabezado principales:

- Un puntero a su clase oculta (Map o HiddenClass) que básicamente define su “forma” o esquema.
- Un puntero al almacén de propiedades nombradas (Properties Store).
- Un puntero al almacén de elementos indexados por entero (Elements Store).

Después del encabezado el objeto aloja sus propiedades directas para un acceso directo sin indirecciones, si el objeto sobrepasa este espacio, las propiedades adicionales se guardan en sus almacenes externos.

La diferencia principal es que mientras un objeto tradicional depende de navegar en una red de punteros, metadatos y cadenas de prototipos, el Float64Array organiza los datos numéricos de manera secuencial dentro de un único bloque contiguo que no tenga metadatos por elemento, esto ayuda a reducir la sobrecarga del recolector de basura (Garbage Collector) lo que permite acceso directo a nivel del CPU.

2. Causa Formal: ¿En qué medida la "forma" o estructura lógica que imponemos a un Tipo de Dato Abstracto (TDA) define y restringe los límites de su eficiencia física en el hardware?

Considero que la forma la define casi por completo. Al seleccionar “un arreglo de coordenadas, con una coordenada como objeto”, estamos imponiendo una estructura de punteros: para llegar a un dato se tiene que seguir referencias hacia otras zonas del montón (heap). La memoria no queda contigua, y el procesador pierde la ventaja de la caché. Una línea de caché típica es de 64 bytes, así que en un Float64Array caben:

$$\frac{64 \text{ bytes}}{8 \text{ bytes/elemento}} = 8 \text{ elementos por línea de caché}$$

Cuando se recorre el arreglo secuencialmente, una sola lectura de memoria trae 8 valores útiles y el prefetcher del hardware puede anticipar las siguientes. Con objetos dispersos, cada acceso puede significar un salto a otra dirección y un posible fallo de caché. La estructura lógica no cambia el resultado del programa, pero sí impone un techo a lo rápido que puede ir en la máquina física. Además, la relación entre ambos enfoques es:

$$\frac{M_{\text{obj}}(N)}{M_{\text{arr}}(N)} = \frac{80N}{16N} = 5$$

O sea que la forma elegida cuesta unas cinco veces más memorias en este caso.

3. Causa Eficiente: Al observar el consumo de memoria, ¿es el intérprete de JavaScript el verdadero artífice (causa eficiente) del desperdicio de recursos, o reside la responsabilidad primaria en la decisión arquitectónica del desarrollador?

La responsabilidad es del desarrollador. El motor V8 únicamente proporciona herramientas y abstracciones dinámicas diseñadas para dar flexibilidad al lenguaje. Si el programador decide instanciar un millón de objetos para alojar simples pares numéricos, está usando una abstracción costosa para un problema de datos masivos. El lenguaje se limita a ejecutar las decisiones de diseño tomadas por el ingeniero.

4. Causa Final: ¿Cuál es el propósito último de abstraer los datos en la programación orientada a objetos si, en su esencia más fundamental, toda optimización crítica requiere retornar a la disposición contigua de bits y bytes en la memoria?

La razón es que sin importar la optimización a realizar, el trabajo de abstraer los datos de manera adecuada, puede ayudar a generar un código que sea no solo más fácil de trabajar para los humanos, sino más fácil de poder pasarse a bits y bytes de memoria. Porque el hecho de obtener una buena abstracción es que puedes trabajar en un código de manera más eficiente y sencilla, menos líneas innecesarias. reutilización de código y generación de código óptimo para que tanto humano como máquina trabajen adecuadamente.

5. Esencia y Existencia: Si dos estructuras de datos (como el Enfoque 1 y el Enfoque 2) resuelven exactamente el mismo problema lógico arrojando los mismos resultados, pero existen físicamente ocupando volúmenes de energía y memoria drásticamente distintos, ¿son en esencia la misma solución? Argumente.

No son la misma solución. Aunque en el plano formal de la lógica pura ambas entreguen los mismos valores, el software existe en un entorno físico con hardware real que consume energía y espacio limitado de RAM. Una alternativa que gasta 300 veces más memoria Heap que la otra impone una huella ambiental y económica distinta, lo que altera sustancialmente la calidad y la esencia de la solución ingenieril.

7. Conclusiones

- Con la presente práctica logramos entender que la forma en que se organiza la información influye directamente en el uso de la memoria. Aunque los objetos y los Float64Array pueden almacenar la misma información, los objetos necesitan más memoria debido a su estructura, por otro lado los arreglos tipados permiten almacenar los datos de una manera más directa y compacta.
- También conseguimos comprender que no siempre la solución que funciona es la más eficiente. Como desarrolladores debemos elegir correctamente las estructuras de datos según la cantidad de información que vamos a manejar. En este caso, Float64Array resulta más adecuado cuando se trabaja con grandes cantidades de datos numéricos y se busca aprovechar mejor los recursos del computador.
- El uso de estructuras contiguas y fuertemente tipadas permite reducir el consumo de memoria en más del 99% frente a los arreglos de objetos

tradicionales, optimizando la gestión del Heap en el motor V8 para grandes volúmenes de información.

- Comprendimos que la eficiencia de un programa no solo depende de su corrección lógica, sino de cómo la estructura de datos seleccionada interactúa con el hardware, la memoria caché de la CPU y la recolección de basura.

7. Bibliografía

1. P. W. Bible and L. Moser, An Open Guide to Data Structures and Algorithms. PALNI Open Press, 2023. [Online]. Available: <https://open.palni.org/>
2. Mozilla Developer Network, "Memory management", MDN Web Docs, 2023. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_Management
3. O. Levin, Discrete Mathematics: An Open Introduction, Autoeditado, 2024.
4. V. Egorov, "Constructor vs. Object.create," *mrle.ph*, 30 de jul. de 2014. [En línea]. Disponible: <https://mrle.ph/blog/2014/07/30/constructor-vs-objectcreate.html>
5. V8 Project Authors, "Field representations and elements kinds," *V8 source repository, docs/objects*. [En línea]. Disponible: <https://chromium.googlesource.com/v8/v8.git/+/main/docs/objects/fields-and-elements.md>
6. C. Bruni, "Fast properties in V8," *V8 Blog*, 2017. [En línea]. Disponible en: <https://v8.dev/blog/fast-properties>
7. V8 Project, "Maps (Hidden Classes) in V8," *V8 Documentation*. [En línea]. Disponible en: <https://v8.dev/docs/hidden-classes>
8. [1] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019 (Revision of IEEE Std 754-2008), pp. 1–84, jul. 2019, doi: 10.1109/IEEESTD.2019.8766229.

8. Anexos

Enlace a GitHub: <https://github.com/B3kk4r/Estructura-de-datos-Grupo9.git>